

Online Judge Platform

Prepared By: Kuswanth Tumma

Software Requirement Specification (SRS)

Online Judge Platform — IEEE Std 830-1998 Standard

| Document Metadata | Details |

| :--- | :--- |

| **Project Title** | Online Judge Platform |

| **Document Standard** | IEEE Std 830-1998 Compliant |

| **Document Version** | 1.0.0-RELEASE |

| **Status** | Formally Approved |

| **Author** | Kuswanth Tumma |

| **Date** | July 2026 |

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document defines the complete functional, non-functional, and system interface requirements for the **Online Judge Platform**. It serves as the formal baseline agreement between stakeholders, product owners, system architects, and development teams.

1.2 Scope

The Online Judge Platform is a web-based automated system that allows registered software engineers and students to read algorithmic problem statements, compose solutions in C++, Java, or Python, and submit their code for automated execution against hidden test suites. The platform handles user authentication, problem administration, asynchronous job execution, containerized sandbox evaluation, submission logging, and global leaderboard calculation.

1.3 Definitions, Acronyms, and Abbreviations

AC: Accepted (All test cases passed cleanly within constraints).

WA: Wrong Answer (Output produced did not match expected testcase output).

TLE: Time Limit Exceeded (Execution exceeded specified milliseconds CPU limit).

MLE: Memory Limit Exceeded (Execution exceeded specified MB RAM limit).

RTE: Runtime Error (Division by zero, segmentation fault, or uncaught exception).

CE: Compilation Error (Syntax or linker error during build phase).

JWT: JSON Web Token.

RBAC: Role-Based Access Control.

cgroups: Linux Kernel Control Groups for resource limits.

seccomp: Linux Kernel Secure Computing Mode system call restriction.

1.4 References

IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

ISO/IEC 27001 Security Standard for Information Technology.

OpenAPI 3.0 Specification Standard.

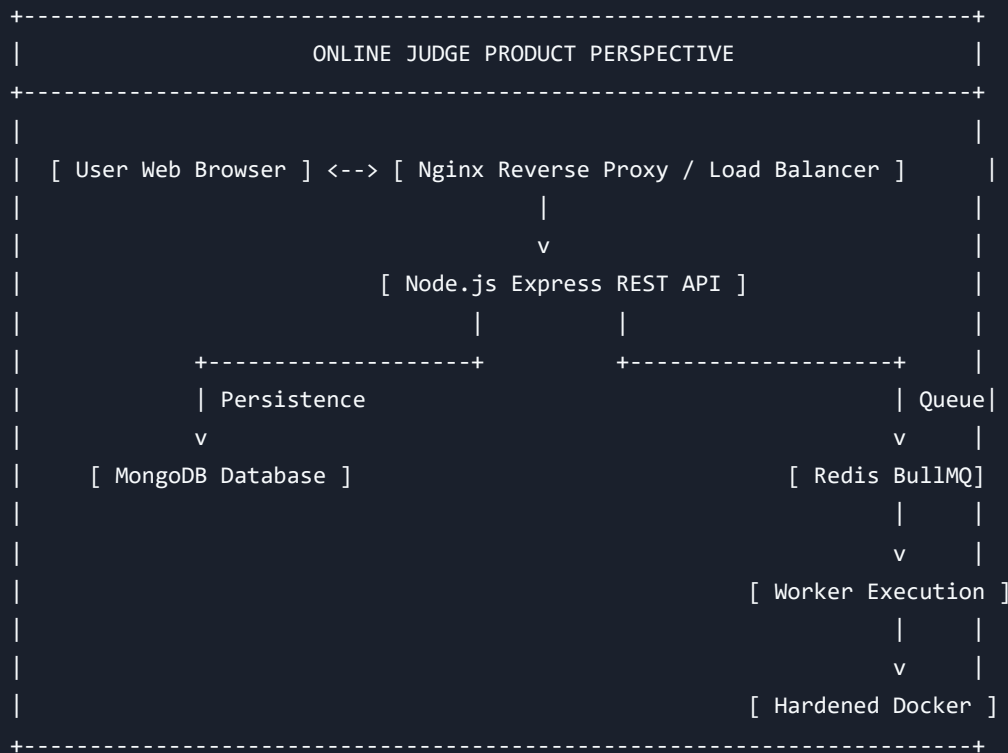
1.5 Overview

The remainder of this document details overall system descriptions, user characteristics, specific functional specifications (categorized by module), non-functional performance benchmarks, and validation acceptance criteria.

2. Overall Description

2.1 Product Perspective

The Online Judge Platform operates as an autonomous multi-tier web system. It relies on a React-based client layer, an Express REST API backend, a Redis message queue for job distribution, a MongoDB instance for persistent state, and Docker engines for code execution.



2.2 Product Functions

User registration, authentication, and session token renewal.

Browse, search, and filter problem statements by difficulty and tags.

Code editor interface supporting syntax highlighting for C++, Python, and Java.

Asynchronous solution execution against concealed test cases.

Real-time verdict delivery with execution time (ms) and peak memory metrics.

Administrator management of problems, test cases, time limits, and user roles.

Global leaderboard computing rankings based on solved counts and submit ratios.

2.3 User Classes and Characteristics

1. **Regular User / Candidate:** Uses the platform to practice coding, submit solutions, track submission history, and view leaderboards. Requires intuitive UI and rapid feedback.

2. **System Administrator / Creator:** Technical user responsible for creating problem statements, managing hidden testcase suites, adjusting constraints, and moderating user accounts.

3. **Automated Worker System:** Unattended background process reading queued execution requests, executing Docker sandboxes, and writing verdict logs.

2.4 Operating Environment

Server OS: Linux (Ubuntu 22.04 LTS / Debian 12 64-bit recommended for native cgroups v2 support).

Runtime Engines: Node.js v18.x+, Docker Engine v24.0+, Redis 7.0+, MongoDB 6.0+.

Client Browsers: Modern web browsers supporting ES6 JavaScript (Chrome 100+, Firefox 100+, Safari 15+, Edge 100+).

2.5 Design and Implementation Constraints

Security Constraint: Arbitrary code execution must be completely isolated from host OS privileges.

Language Compatibility: Must support GCC 11+ (C++17/20), Python 3.10+, and OpenJDK 17+.

NoSQL Schema Constraint: Data models must handle dynamic arrays of testcases without performance degradation.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Responsive web UI built with React and Tailwind CSS.

Code Editor component integrated with Monaco Editor supporting line numbers, auto-indentation, and syntax highlighting.

Modal dialogs displaying compilation errors and execution stack traces.

3.1.2 Hardware Interfaces

Multi-core x86_64 host CPU supporting hardware virtualization and cgroups v2.

Minimum 8 GB RAM on Worker nodes to support parallel Docker container execution.

3.1.3 Software Interfaces

Docker Engine API: Invoked via CLI bindings to create, execute, and destroy sandboxes.

MongoDB Driver: Mongoose ORM for database connection pooling and schema enforcement.

3.1.4 Communications Interfaces

Secure HTTPS communication using TLS 1.3 encryption.

JSON data transfer format for REST API request and response payloads.

3.2 Detailed Functional Requirements

Module 1: Authentication & Authorization

FR-1.1: The system shall allow users to register with name, unique email, and password.

FR-1.2: Passwords must be hashed using `bcrypt` with a minimum salt factor of 12 prior to database storage.

FR-1.3: The system shall issue a signed JWT upon successful authentication.

FR-1.4: The system shall reject requests to restricted endpoints if the JWT is invalid or expired.

Module 2: Problem Management

FR-2.1: System administrators shall be able to create, read, update, and soft-delete problems.

FR-2.2: Each problem record shall contain title, slug, statement text, difficulty rating, time limit (ms), and memory limit (MB).

FR-2.3: Regular users shall be able to search problems by keyword and filter by difficulty.

Module 3: Test Case Management

FR-3.1: Administrators shall be able to upload multiple input/expected-output test pairs for any problem.

FR-3.2: The system shall distinguish between public sample test cases and hidden evaluation test cases.

FR-3.3: Hidden testcase outputs shall never be exposed to regular users in raw API responses.

Module 4: Code Submission & Queue Processing

FR-4.1: Regular users shall submit source code up to 64 KB in size for any published problem.

FR-4.2: The system shall enqueue submission tasks into a Redis BullMQ queue and assign an initial state of `Pending`.

FR-4.3: Workers shall pull jobs concurrently and set status to `Processing`.

Module 5: Sandbox Execution & Verdict Evaluation

FR-5.1: The system shall instantiate isolated Docker containers with non-root user privilege (`UID 10001`).

FR-5.2: The system shall enforce CPU quotas, memory limits, and disable network interfaces (`--network none`).

FR-5.3: The system shall compare program output against expected output using whitespace-trimmed diffing.

FR-5.4: The system shall compute one of six final verdicts: `AC`, `WA`, `TLE`, `MLE`, `RTE`, or `CE`.

Module 6: Leaderboard & History

FR-6.1: The system shall maintain an immutable submission history per user.

FR-6.2: The system shall aggregate and publish a global leaderboard sorted by solved count and submit accuracy.

3.3 Non-Functional & System Performance Attributes

1. **Response Time SLA:** Non-submission REST endpoints must respond within 150 milliseconds for 95% of requests under nominal load.

2. **Evaluation Throughput:** The worker pool must evaluate at least 1,000 code submissions per minute under auto-scaled worker node configurations.

3. **Availability:** The platform must achieve 99.9% uptime per calendar month.

4. **Security & Data Isolation:** Zero kernel leaks or unauthorized host read access from within Docker containers.

4. Verification & Validation Acceptance Criteria

Requirement ID	Verification Method	Acceptance Test Criteria
----------------	---------------------	--------------------------

:---	:---	:---
------	------	------

| **FR-1.2** | Automated Security Test | Inspect MongoDB document records to confirm passwords are stored as `$2b$12$...` bcrypt hashes. |

| **FR-4.1** | Boundary Unit Test | Submit a 65 KB source code payload and verify HTTP 400 Bad Request error returned. |

| **FR-5.2** | Penetration / Sandbox Test| Execute code attempting socket creation (`socket()`); verify system returns `Runtime Error` and network access fails. |

| **FR-5.3** | Automated Diff Test | Execute code returning extra trailing spaces; verify output normalization evaluates verdict to `Accepted` . |

| **FR-5.4** | Time Limit Test | Submit `while(true){}` ; verify container is forcibly terminated after CPU limit + 1s and verdict equals `TLE` . |